

LC-EKF VIO 제로베이스 구축 매뉴얼

ROS 없이 C++20, OpenCV, Eigen, yaml-cpp로 빌드하고 실행하기

D:/02_research/04_cpp_LC-EKF_VIO

2026-05-01

요 약

이 문서는 D:/02_research/04_cpp_LC-EKF_VIO 폴더의 LC-EKF VIO 시스템을 빈 Windows/WSL 환경에서 다시 구축하기 위한 절차서다. 대상 독자는 저장소를 처음 받은 사람이며, 목표는 의존성 설치, 데이터 배치, YAML 설정, CMake 빌드, 단일 실행, 결과 검증까지 재현 가능한 상태로 만드는 것이다. 본 시스템은 ROS, catkin, colcon 없이 순수 CMake 기반으로 동작한다.

차 례

1	매뉴얼 범위	3
1.1	목표 산출물	3
1.2	시스템 구성	3
2	수학적 방법론	5
2.1	좌표계와 변환 convention	5
2.2	상태와 오차 상태	6
2.3	IMU 측정 모델과 공칭 상태 전파	6
2.4	초기 정렬	7
2.5	15-state 공분산 전파	7
2.6	Stereo VO 측정과 world anchoring	8
2.7	LC-EKF 위치 업데이트	9
2.8	IMU-only 설정의 의미	10
2.9	ATE RMSE 계산	10
3	사전 준비	12
3.1	Windows와 WSL	12
3.2	시스템 패키지 설치	12
3.3	저장소 구조 확인	13
4	데이터셋 배치	14
4.1	KITTI raw	14
4.2	EuRoC	15
5	YAML 설정	16
5.1	경로 정책	16
5.2	필수 키	16
5.3	KITTI 캘리브레이션 추출	17
5.4	최소 KITTI YAML 예시	18
6	빌드	20

6.1	일반 빌드	20
6.2	Rerun 선택 빌드	20
6.3	빌드 실패 빠른 판별	20
7	실행	22
7.1	단일 KITTI 실행	22
7.2	단일 EuRoC 실행	22
7.3	프레임 수 제한 테스트	22
7.4	Rerun 기록 생성	23
8	결과 검증과 시각화	24
8.1	출력 파일	24
8.2	plot 생성	24
8.3	정상/비정상 판정	24
9	운영 절차 체크리스트	26
9.1	처음 구축할 때	26
9.2	새 KITTI drive를 추가할 때	26
9.3	새 개발자가 가장 먼저 볼 파일	26
10	트러블슈팅	28
10.1	실행 전 오류	28
10.2	실행 후 성능 문제	28
10.3	경로 문제를 빠르게 찾는 명령	28
11	부록: 한 번에 따라 하는 명령	30
11.1	EuRoC V2_02 빠른 실행	30
11.2	EuRoC V2_02 IMU-only 실행	30
11.3	EuRoC V2_02 Rerun 기록	31
11.4	Rerun viewer 팁	31
11.5	KITTI 기본 실행	32
11.6	Rerun 기록 포함 실행	32
11.7	최종 확인	32

제 1 장

매뉴얼 범위

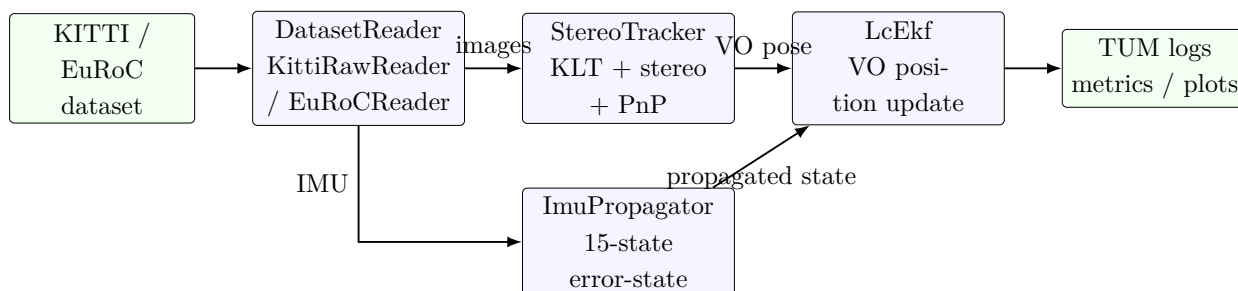
1.1 목표 산출물

제로베이스 구축이 끝나면 다음 상태가 되어야 한다.

항목	완료 기준
빌드 산출물	build/run_vio, build/run_euroc 실행 파일 생성
설정 파일	config/*.yaml의 dataset, output, T_cam_imu가 현재 환경과 일치
데이터셋	KITTI raw 또는 EuRoC 디렉터리가 reader가 기대하는 구조로 배치
실행 결과	results/ <run_name>/ trajectory_tum.txt, metrics.txt, state_log.txt 생성
시각화	trajectory_plot.png, state_plot.png 생성

기준 환경: 이 프로젝트의 표준 빌드 경로는 WSL Ubuntu이다. Windows 네이티브 빌드도 이론상 가능하지만 OpenCV, Eigen, yaml-cpp 설치 방식이 환경마다 달라 재현성이 낮다. 따라서 본 매뉴얼은 Windows 파일시스템에 저장소를 두고 WSL에서 빌드하는 방식을 기준으로 한다.

1.2 시스템 구성



핵심 타겟은 run_vio 하나다. dataset_type이 euroc이면 EuRoC reader를, kitti_raw이면 KITTI

raw reader를 사용한다.

제 2 장

수학적 방법론

2.1 좌표계와 변환 convention

이 프로젝트의 수식은 코드의 변수명과 맞추어 쓴다. W 는 world frame, I 는 IMU/body frame, C 는 cam0 frame이다. 회전 R_{AB} 는 B frame 벡터를 A frame으로 보낸다.

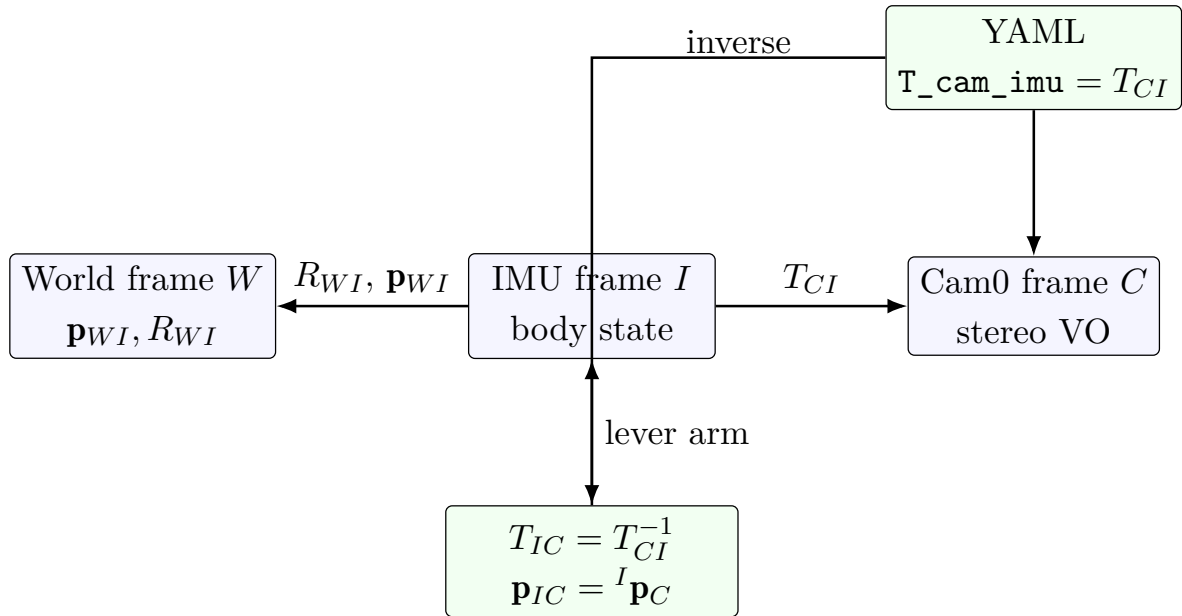


그림 2.1: 코드와 YAML이 사용하는 좌표계. EKF 상태는 $W \leftarrow I$, YAML calibration은 $C \leftarrow I$, update lever arm은 $I \leftarrow C$ 를 사용한다.

$${}^A\mathbf{p} = R_{AB} {}^B\mathbf{p} + {}^A\mathbf{p}_B. \quad (2.1)$$

YAML의 $T_{\text{cam_imu}}$ 는 T_{CI} , 즉 IMU 좌표계의 점을 camera 좌표계로 보내는 변환이다. LC-EKF 업데이트에서는 cam0 origin의 IMU 좌표가 필요하므로 $T_{IC} = T_{CI}^{-1}$ 를 사용하고,

$$\mathbf{p}_{IC} = {}^I\mathbf{p}_C = (T_{IC})_{0:3,3} \quad (2.2)$$

를 lever arm으로 둔다. 코드에서는 LcEkf 생성자에서 이 값을 계산한다.

기호	의미
$\mathbf{p}_{WI}$	world frame에서 본 IMU 위치
$\mathbf{v}_{WI}$	world frame에서 본 IMU 속도
R_{WI}	IMU 벡터를 world 벡터로 보내는 자세
$\mathbf{b}_g, \mathbf{b}_a$	gyro/accel bias
$\mathbf{p}_{IC}$	IMU frame에서 본 cam0 origin 위치
$\mathbf{z}_{VO}$	stereo VO가 제공하는 cam0 위치 측정값

2.2 상태와 오차 상태

공칭 상태는 위치, 속도, 자세, bias로 구성된다.

$$\mathbf{x} = \{\mathbf{p}_{WI}, \mathbf{v}_{WI}, R_{WI}, \mathbf{b}_g, \mathbf{b}_a\}. \quad (2.3)$$

EKF 공분산은 회전 행렬 자체가 아니라 작은 회전 오차 $\delta\theta$ 를 포함하는 15차원 error-state에 대해 유지한다.

$$\delta\mathbf{x} = [\delta\mathbf{p}^\top \ \delta\mathbf{v}^\top \ \delta\theta^\top \ \delta\mathbf{b}_g^\top \ \delta\mathbf{b}_a^\top]^\top \in \mathbb{R}^{15}. \quad (2.4)$$

자세 보정은 SO(3)의 exponential map으로 적용한다.

$$R_{WI} \leftarrow R_{WI} \text{Exp}(\delta\theta). \quad (2.5)$$

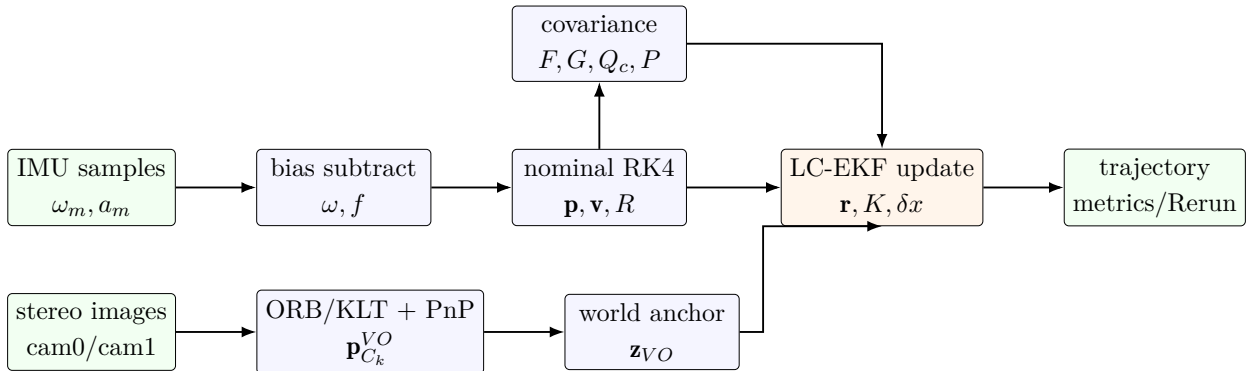


그림 2.2: 수학적 처리 흐름. IMU는 예측을 만들고, stereo VO는 cam0 위치 측정값을 만들며, LC-EKF는 두 정보를 위치 update로 결합한다.

2.3 IMU 측정 모델과 공칭 상태 전파

gyro와 accelerometer raw 측정값을 다음처럼 둔다.

$$\boldsymbol{\omega} = \boldsymbol{\omega}_m - \mathbf{b}_g - \mathbf{n}_g, \quad (2.6)$$

$$\mathbf{f} = \mathbf{a}_m - \mathbf{b}_a - \mathbf{n}_a. \quad (2.7)$$

여기서 $\mathbf{f}$ 는 IMU frame의 specific force이다. 공칭 상태 동역학은 다음과 같다.

$$\dot{\mathbf{p}}_{WI} = \mathbf{v}_{WI}, \quad (2.8)$$

$$\dot{\mathbf{v}}_{WI} = R_{WI}\mathbf{f} + \mathbf{g}_W, \quad (2.9)$$

$$\dot{R}_{WI} = R_{WI}[\boldsymbol{\omega}]_{\times}, \quad (2.10)$$

$$\dot{\mathbf{b}}_g = \mathbf{0}, \quad (2.11)$$

$$\dot{\mathbf{b}}_a = \mathbf{0}. \quad (2.12)$$

구현에서는 bias를 뺀 $\boldsymbol{\omega}$, $\mathbf{f}$ 를 사용해 위치와 속도를 RK4로 적분하고, 자세는

$$R_{k+1} = R_k \text{Exp}(\Delta t \boldsymbol{\omega}) \quad (2.13)$$

로 갱신한 뒤 quaternion normalization으로 다시 정규직교화한다.

2.4 초기 정렬

초기화는 GT를 쓰지 않고 첫 카메라 timestamp 이후 `init.static_window_sec` 구간의 IMU 평균으로 수행한다.

$$\bar{\boldsymbol{\omega}} = \frac{1}{N} \sum_{i=1}^N \boldsymbol{\omega}_{m,i}, \quad (2.14)$$

$$\bar{\mathbf{a}} = \frac{1}{N} \sum_{i=1}^N \mathbf{a}_{m,i}. \quad (2.15)$$

초기 gyro bias는 $\mathbf{b}_{g,0} = \bar{\boldsymbol{\omega}}$ 로 둔다. 초기 자세는 정지 상태에서

$$R_{WI,0}\bar{\mathbf{a}} \approx \begin{bmatrix} 0 & 0 & g \end{bmatrix}^{\top}, \quad \mathbf{g}_W = \begin{bmatrix} 0 & 0 & -g \end{bmatrix}^{\top} \quad (2.16)$$

가 되도록 $\bar{\mathbf{a}}$ 를 world +Z 방향에 맞춘다. 이 조건이면 정지 상태에서 $R_{WI}\mathbf{f} + \mathbf{g}_W \approx \mathbf{0}$ 이 된다.

2.5 15-state 공분산 전파

오차 상태의 연속시간 근사는 다음 꼴이다.

$$\delta \dot{\mathbf{x}} = F\delta \mathbf{x} + G\mathbf{n}, \quad \mathbf{n} = \begin{bmatrix} \mathbf{n}_g^{\top} & \mathbf{n}_a^{\top} & \mathbf{n}_{wg}^{\top} & \mathbf{n}_{wa}^{\top} \end{bmatrix}^{\top}. \quad (2.17)$$

구현에서 사용하는 주요 F block은 다음과 같다.

$$F_{p,v} = I, \quad (2.18)$$

$$F_{v,\theta} = -R_{WI}[\mathbf{f}]_{\times}, \quad F_{v,b_a} = -R_{WI}, \quad (2.19)$$

$$F_{\theta,\theta} = -[\boldsymbol{\omega}]_{\times}, \quad F_{\theta,b_g} = -R_{WI}. \quad (2.20)$$

noise injection G 는 accel noise가 속도 오차로, gyro noise가 자세 오차로, bias random walk가 각각 bias 오차로 들어가도록 구성한다.

$$G_{v,n_a} = R_{WI}, \quad G_{\theta,n_g} = R_{WI}, \quad (2.21)$$

$$G_{b_g,n_{wg}} = I, \quad G_{b_a,n_{wa}} = I. \quad (2.22)$$

연속 noise density는 YAML의 imu_noise 값으로 만든다.

$$Q_c = \text{diag}(\sigma_g^2 I, \sigma_a^2 I, \sigma_{bg}^2 I, \sigma_{ba}^2 I). \quad (2.23)$$

이산화는 1차 Euler 근사다.

$$\Phi \approx I + F\Delta t, \quad (2.24)$$

$$P_{k+1} = \Phi P_k \Phi^\top + G Q_c G^\top \Delta t. \quad (2.25)$$

따라서 공칭 상태는 RK4로 비교적 정교하게 적분하지만, 공분산은 단순 1차 근사로 전파한다.

2.6 Stereo VO 측정과 world anchoring

Stereo VO는 첫 cam0 frame을 local world로 두고 상대 camera pose를 추정한다. 초기 frame에서는 rectified stereo pair에서 ORB matching을 수행하고, disparity로 깊이를 얻는다.

$$Z = \frac{f_x b}{d}, \quad (2.26)$$

$$X = \frac{(u - c_x)Z}{f_x}, \quad (2.27)$$

$$Y = \frac{(v - c_y)Z}{f_y}. \quad (2.28)$$

이후 frame에서는 좌측 이미지 KLT tracking으로 2D-3D 대응을 유지하고, PnP RANSAC으로 camera pose를 구한다.

$$\min_{R,t} \sum_i \rho \left(\|\pi(K(R\mathbf{P}_i + t)) - \mathbf{u}_i\|^2 \right). \quad (2.29)$$

VO local 좌표계의 camera 위치 $\mathbf{p}_{C_k}^{VO}$ 는 첫 cam0 pose를 기준으로 EKF world에 anchor된다.

$$R_{WC_0} = R_{WI,0}R_{IC}, \quad (2.30)$$

$$\mathbf{p}_{WC_0} = R_{WI,0}\mathbf{p}_{IC} + \mathbf{p}_{WI,0}, \quad (2.31)$$

$$\mathbf{p}_{C_k}^W = R_{WC_0}\mathbf{p}_{C_k}^{VO} + \mathbf{p}_{WC_0}. \quad (2.32)$$

이 $\mathbf{p}_{C_k}^W$ 가 LC-EKF의 위치 측정값으로 들어간다.

2.7 LC-EKF 위치 업데이트

측정값은 world frame에서 본 cam0 origin 위치다.

$$\mathbf{z}_{VO} = \mathbf{p}_C^W. \quad (2.33)$$

현재 EKF 상태로 예측한 cam0 위치는 IMU 위치와 lever arm으로 계산한다.

$$h(\mathbf{x}) = \mathbf{p}_{WI} + R_{WI}\mathbf{p}_{IC}. \quad (2.34)$$

innovation은

$$\mathbf{r} = \mathbf{z}_{VO} - h(\mathbf{x}) \quad (2.35)$$

이고, 측정 Jacobian의 non-zero block은 다음 두 개다.

$$H_p = I, \quad (2.36)$$

$$H_\theta = -R_{WI}[\mathbf{p}_{IC}]_\times. \quad (2.37)$$

VO 측정 noise는 scalar 설정 σ_{VO} 로부터

$$R_{VO} = \sigma_{VO}^2 I \quad (2.38)$$

로 둔다. Kalman update는 표준식과 Joseph form 공분산 갱신을 사용한다.

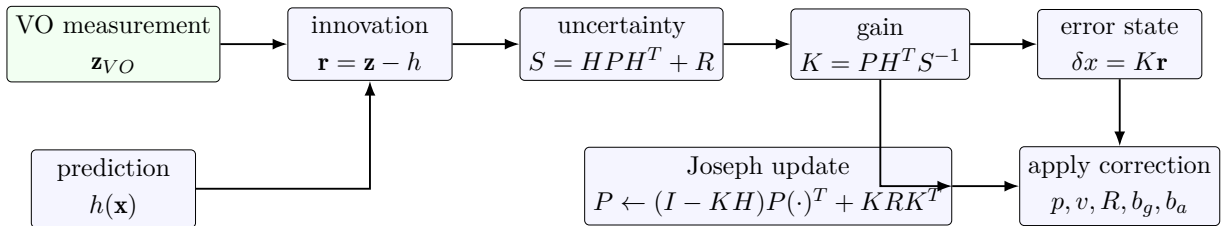


그림 2.3: LC-EKF 위치 업데이트 내부 흐름. 측정 residual로 error-state를 계산한 뒤 공칭 상태와 공분산을 함께 보정한다.

$$S = HPH^\top + R_{VO}, \quad (2.39)$$

$$K = PH^\top S^{-1}, \quad (2.40)$$

$$\delta \mathbf{x} = K \mathbf{r}, \quad (2.41)$$

$$P \leftarrow (I - KH)P(I - KH)^\top + KR_{VO}K^\top. \quad (2.42)$$

상태 보정은 다음처럼 적용한다.

$$\mathbf{p} \leftarrow \mathbf{p} + \delta \mathbf{p}, \quad (2.43)$$

$$\mathbf{v} \leftarrow \mathbf{v} + \delta \mathbf{v}, \quad (2.44)$$

$$R \leftarrow R \text{Exp}(\delta \boldsymbol{\theta}), \quad (2.45)$$

$$\mathbf{b}_g \leftarrow \mathbf{b}_g + \delta \mathbf{b}_g, \quad (2.46)$$

$$\mathbf{b}_a \leftarrow \mathbf{b}_a + \delta \mathbf{b}_a. \quad (2.47)$$

이 구조가 **loose coupling**인 이유는 raw feature residual을 EKF가 직접 쓰지 않고, frontend가 이미 계산한 VO camera 위치만 측정값으로 쓰기 때문이다. 구현은 단순하지만 VO outlier나 scale/depth 오류가 위치 측정값에 그대로 들어올 수 있다.

2.8 IMU-only 설정의 의미

`config/euroc_v202_imuonly.yaml`은 별도의 전파-only 실행기를 쓰는 것이 아니라 $\sigma_{VO} = 10^9$ 로 설정한다. 따라서

$$R_{VO} = \sigma_{VO}^2 I \quad (2.48)$$

가 매우 커지고,

$$K = PH^\top (HPH^\top + R_{VO})^{-1} \approx 0 \quad (2.49)$$

가 된다. 결과적으로 VO update가 실행되더라도 EKF 상태 보정량은 거의 0이므로, trajectory는 IMU propagation에 의해 지배된다.

2.9 ATE RMSE 계산

결과 검증의 ATE는 추정 위치열 $\mathbf{p}_i^{est}$ 와 GT 위치열 $\mathbf{p}_i^{gt}$ 사이의 rigid alignment를 먼저 구한 뒤 계산한다. 코드는 Eigen의 `umeyama(..., false)`를 사용하므로 scale은 고정하고 회전과 평행이동만 맞춘다.

$$T^* = \arg \min_{T \in SE(3)} \sum_i \left\| T \mathbf{p}_i^{est} - \mathbf{p}_i^{gt} \right\|^2. \quad (2.50)$$

ATE RMSE는 다음과 같다.

$$ATE_{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N \left\| T^* \mathbf{p}_i^{est} - \mathbf{p}_i^{gt} \right\|^2}. \quad (2.51)$$

따라서 `metrics.txt`의 ATE는 절대 좌표 원점 차이보다 trajectory 모양과 누적 오차를 비교하는 지표에 가깝다.

제 3 장

사전 준비

3.1 Windows와 WSL

Windows PowerShell에서 WSL Ubuntu가 있는지 확인한다.

```
wsl -l -v
```

Listing 3.1: PowerShell에서 WSL 확인

Ubuntu가 없다면 Microsoft Store 또는 관리자 PowerShell에서 WSL2 Ubuntu를 설치한다. 설치 후 Ubuntu 터미널을 열고 다음부터는 WSL 셸에서 작업한다.

```
cd /mnt/d/02_research/04_cpp_LC-EKF_VIO
pwd
```

Listing 3.2: WSL에서 저장소 위치로 이동

YAML 설정에는 Windows 경로 D:\...를 넣지 않는다. WSL에서 실행할 때는 /mnt/d/... 또는 저장소 루트 기준 상대 경로를 사용한다.

3.2 시스템 패키지 설치

처음 세팅하는 Ubuntu에서 다음 패키지를 설치한다.

```
sudo apt update
sudo apt install -y \
  build-essential cmake git pkg-config unzip \
  libeigen3-dev libopencv-dev libyaml-cpp-dev \
  python3 python3-numpy python3-matplotlib
```

Listing 3.3: 필수 빌드 및 실행 의존성

버전 확인:

```
g++ --version
cmake --version
pkg-config --modversion opencv4
python3 - <<'PY'
import numpy, matplotlib
```

```
print("numpy", numpy.__version__)
print("matplotlib", matplotlib.__version__)
PY
```

Listing 3.4: 도구 버전 확인

3.3 저장소 구조 확인

소스 폴더가 정상이라면 루트에 다음 항목이 있어야 한다.

```
apps/          # run_vio.cpp
config/        # dataset and calibration YAML
docs/          # manuals and generated PDF
include/       # public headers
src/           # implementation
tools/         # plot_trajectory.py
CMakeLists.txt
README.md
```

Listing 3.5: 루트 구조

이 프로젝트는 `build/`, `build_rerun/`, `results/`, `logs/` 같은 생성물을 포함할 수 있다. 제로베이스 환경에서는 없어도 된다.

제 4 장

데이터셋 배치

4.1 KITTI raw

KITTI raw를 사용할 때 reader가 기대하는 최소 구조는 다음과 같다.

```
data/kitti_raw/ └──
2011_09_26/
├── calib_cam_to_cam.txt
├── calib_imu_to_velo.txt
├── calib_velo_to_cam.txt
└── 2011_09_26_drive_0117_sync/
    ├── image_00/
    │   ├── data/*.png
    │   └── timestamps.txt
    ├── image_01/
    │   ├── data/*.png
    │   └── timestamps.txt
    └── oxts/
        ├── data/*.txt
        ├── timestamps.txt
        └── dataformat.txt
```

Listing 4.1: KITTI raw 권장 배치

흑백 스테레오 폴더는 image_00/image_01이다. reader는 우선 이 쌍을 찾고, 없을 때만 컬러 페어인 image_02/image_03을 찾는다. OXTS는 IMU와 ground truth 계산에 사용된다.

KITTI 체크리스트

- 날짜 디렉터리 바로 아래에 calib 3종이 있는가.
- drive 디렉터리 이름이 *_sync로 끝나는가.
- image_00/data와 image_01/data에 같은 개수의 PNG가 있는가.
- oxts/data/*.txt와 oxts/timestamps.txt가 있는가.

4.2 EuRoC

EuRoC reader는 root 바로 아래에 다음 폴더가 있다고 가정한다.

```
<dataset_root>/ |——  
imu0/data.csv |——  
cam0/data.csv |——  
cam0/data/*.png |——  
cam1/data.csv |——  
cam1/data/*.png |——  
state_groundtruth_estimate0/data.csv
```

Listing 4.2: EuRoC reader가 기대하는 구조

공식 EuRoC ASL 압축을 풀면 보통 MH_01_easy/mav0/... 형태가 된다. 그 경우 YAML의 dataset은 MH_01_easy가 아니라 MH_01_easy/mav0 까지 지정해야 한다.

제 5 장

YAML 설정

5.1 경로 정책

제로베이스에서 가장 흔한 실패는 오래된 절대 경로다. 현재 저장소에 포함된 일부 KITTI 설정은 과거 루트인 `/mnt/d/02_research/06_cpp_LC-EKF_VIO_kitti_raw`를 가리킬 수 있다. 이 문서의 대상 루트는 다음과 같다.

```
/mnt/d/02_research/04_cpp_LC-EKF_VIO
```

저장소 루트에서 실행한다면 상대 경로가 가장 덜 깨진다.

```
dataset_type: "kitti_raw"
dataset: "data/kitti_raw/2011_09_26/2011_09_26_drive_0117_sync"
output: "results/kitti_raw_2011_09_26_drive_0117"
max_frames: 0
```

Listing 5.1: 권장 경로 설정

만약 `cd build` 안에서 실행한다면 상대 경로 기준이 바뀌므로 다음 중 하나를 선택한다.

- 저장소 루트에서만 실행한다: `./build/run_vio config/...yaml`
- YAML 경로를 절대 WSL 경로로 쓴다: `/mnt/d/02_research/04_cpp_LC-EKF_VIO/...`
- build 디렉터리 기준 상대 경로로 바꾼다: `../data/..., ../results/...`

5.2 필수 키

키	의미
<code>dataset_type</code>	euroc 또는 kitti_raw
<code>dataset</code>	reader가 읽을 데이터셋 root
<code>output</code>	로그와 trajectory 파일을 저장할 디렉터리
<code>max_frames</code>	0이면 전체, 양수면 해당 프레임 수까지만 실행
<code>init.static_window_sec</code>	초기 IMU 정지구간 평균 시간
<code>imu_noise</code>	gyro/accel white noise와 random walk

ekf.sigma_vo
cam0, cam1

VO 위치 측정 잡음. 작을수록 VO를 더 신뢰
intrinsics, distortion, T_cam_imu

5.3 KITTI 캘리브레이션 추출

KITTI raw의 calib 3종에서 이 프로젝트가 요구하는 것은 rectified cam0/cam1 기준 T_{cam_imu} 두 개다. 의미는 IMU 좌표계의 점을 camera 좌표계로 보내는 T_{CI} 이다.

$$T_{cam0,imu} = R_{rect,00} T_{cam,velo} T_{velo,imu}$$
$$T_{cam1,imu} = T_{cam1,cam0} T_{cam0,imu}$$

다음 스크립트를 임시 파일로 저장해 실행하면 intrinsics와 두 행렬을 확인할 수 있다.

```
from pathlib import Path
import numpy as np

CALIB = Path("/mnt/d/02_research/04_cpp_LC-EKF_VIO/data/kitti_raw/2011_09_26")

def parse_kv(path):
    out = {}
    for line in path.read_text().splitlines():
        if ":" not in line:
            continue
        k, v = line.split(":", 1)
        out[k.strip()] = np.fromstring(v, sep=" ")
    return out

def homog(R, t):
    T = np.eye(4)
    T[:3, :3] = R.reshape(3, 3)
    T[:3, 3] = t.reshape(3)
    return T

cam = parse_kv(CALIB / "calib_cam_to_cam.txt")
v2c = parse_kv(CALIB / "calib_velo_to_cam.txt")
i2v = parse_kv(CALIB / "calib_imu_to_velo.txt")

T_cam_velo = homog(v2c["R"], v2c["T"])
T_velo_imu = homog(i2v["R"], i2v["T"])
R_rect_00 = np.eye(4)
R_rect_00[:3, :3] = cam["R_rect_00"].reshape(3, 3)

T_cam0_imu = R_rect_00 @ T_cam_velo @ T_velo_imu

P_rect_00 = cam["P_rect_00"].reshape(3, 4)
P_rect_01 = cam["P_rect_01"].reshape(3, 4)
fx, fy = P_rect_00[0, 0], P_rect_00[1, 1]
cx, cy = P_rect_00[0, 2], P_rect_00[1, 2]
baseline_x = -P_rect_01[0, 3] / fx

T_cam1_cam0 = np.eye(4)
```

```

T_cam1_cam0[0, 3] = -baseline_x
T_cam1_imu = T_cam1_cam0 @ T_cam0_imu

print(f"width height: {cam['S_rect_00'][0]:.0f} {cam['S_rect_00'][1]:.0f}")
print(f"fx fy cx cy: {fx:.4f} {fy:.4f} {cx:.4f} {cy:.4f}")
print(f"baseline_x: {baseline_x:.6f}")
for name, T in [("cam0", T_cam0_imu), ("cam1", T_cam1_imu)]:
    print(name)
    for row in T:
        print(" - [ " + ", ".join(f"{v: .12f}" for v in row) + " ]")

```

Listing 5.2: KITTI calib에서 YAML 값 추출

KITTI rectified 이미지를 기준으로 쓸 때 왜곡 계수 k_1 , k_2 , p_1 , p_2 는 보통 0으로 둔다. R_{rect_00} 적용을 빠뜨리면 intrinsics와 좌표계가 어긋나 trajectory가 크게 발산한다.

5.4 최소 KITTI YAML 예시

```

dataset_type: "kitti_raw"
dataset: "data/kitti_raw/2011_09_26/2011_09_26_drive_0117_sync"
output: "results/kitti_raw_2011_09_26_drive_0117"
max_frames: 0

init:
  mode: "stationary_imu"
  static_window_sec: 1.0
  gravity_norm: 9.81

imu_noise:
  gyro: 1.6968e-4
  accel: 2.0000e-3
  gyro_walk: 1.9393e-5
  accel_walk: 3.0000e-3

ekf:
  sigma_vo: 0.10

cam0:
  width: 1242
  height: 375
  fx: 721.5377
  fy: 721.5377
  cx: 609.5593
  cy: 172.8540
  k1: 0.0
  k2: 0.0
  p1: 0.0
  p2: 0.0
  T_cam_imu:
    - [ 0.000998747206, -0.999990382071, 0.004259378488, -0.314076869806 ]
    - [ 0.008416901828, -0.004250821136, -0.999955569675, 0.719452035633 ]
    - [ 0.999964048697, 0.001034553284, 0.008412575209, -1.089082940192 ]
    - [ 0.0, 0.0, 0.0, 1.0 ]

```

```
cam1:
  width: 1242
  height: 375
  fx: 721.5377
  fy: 721.5377
  cx: 609.5593
  cy: 172.8540
  k1: 0.0
  k2: 0.0
  p1: 0.0
  p2: 0.0
  T_cam_imu:
    - [ 0.000998748817, -0.999990390387, 0.004259383056, -0.851244549356 ]
    - [ 0.008416901410, -0.004250823329, -0.999955516666, 0.719451996647 ]
    - [ 0.999964103141, 0.001034555313, 0.008412574849, -1.089083003993 ]
    - [ 0.0, 0.0, 0.0, 1.0 ]
```

Listing 5.3: 저장소 루트 실행 기준 최소 예시

제 6 장

빌드

6.1 일반 빌드

WSL에서 저장소 루트로 이동한 뒤 configure와 build를 분리해 실행한다.

```
cd /mnt/d/02_research/04_cpp_LC-EKF_VIO
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j"${nproc}"
```

Listing 6.1: Release 빌드

성공하면 다음 파일이 생긴다.

```
build/run_vio
build/run_euroc
```

run_euroc는 과거 명령 호환용 이름이며 내부적으로 apps/run_vio.cpp와 같은 main을 사용한다.

6.2 Rerun 선택 빌드

Rerun은 선택 기능이다. 일반 빌드는 Rerun SDK 없이 동작한다. Rerun 기록을 만들고 싶을 때만 별도 build 디렉터리를 사용한다.

```
cd /mnt/d/02_research/04_cpp_LC-EKF_VIO
cmake -S . -B build_rerun -DCMAKE_BUILD_TYPE=Release -DENABLE_RERUN=ON
cmake --build build_rerun -j"${nproc}"
```

Listing 6.2: Rerun 활성화 빌드

ENABLE_RERUN=ON은 CMake FetchContent로 Rerun C++ SDK zip을 내려받는다. 네트워크가 막힌 환경에서는 일반 빌드로 먼저 검증한 뒤, 사내/학교 네트워크 정책에 맞춰 SDK 캐시를 준비한다.

6.3 빌드 실패 빠른 판별

증상	조치
Eigen3Config.cmake not found	<code>sudo apt install libeigen3-dev</code>
OpenCVConfig.cmake not found	<code>sudo apt install libopencv-dev</code> 후 <code>pkg-config --modversion opencv4</code> 확인
yaml-cpp not found	<code>sudo apt install libyaml-cpp-dev</code>
CMAKE_CXX_STANDARD 20 관련 오류	Ubuntu의 g++가 너무 오래됨. Ubuntu 22.04 이상 권장
Rerun 다운로드 실패	<code>-DENABLE_RERUN=OFF</code> 일반 빌드로 우선 진행

제 7 장

실행

7.1 단일 KITTI 실행

항상 저장소 루트에서 실행하는 방식을 권장한다.

```
cd /mnt/d/02_research/04_cpp_LC-EKF_VIO
./build/run_vio config/kitti_raw_2011_09_26_drive_0117.yaml
```

Listing 7.1: KITTI 실행

실행 마지막에 다음 형태가 나오면 정상이다.

```
=====
Frames processed : 660
ATE RMSE          : 152.955 cm
Output            : results/kitti_raw_2011_09_26_drive_0117
=====
```

Listing 7.2: 정상 종료 예시

이 저장소의 기존 결과 기준으로 drive_0001은 108 frames, ATE 약 156 cm, drive_0117은 660 frames, ATE 약 153 cm 수준이다. LC-EKF 구조상 cm급 정밀도보다 미터급 누적 오차가 자연스럽다.

7.2 단일 EuRoC 실행

EuRoC는 YAML의 dataset 경로가 실제 mav0 구조를 가리키는지 먼저 확인한다.

```
cd /mnt/d/02_research/04_cpp_LC-EKF_VIO
./build/run_vio config/euroc_v101.yaml
```

Listing 7.3: EuRoC 실행

7.3 프레임 수 제한 테스트

처음 환경 검증만 할 때는 YAML의 max_frames를 50 또는 100으로 바꿔 빠르게 확인한다. 전체 실행 전 다음 항목만 보면 된다.

- No camera data loaded, No IMU data loaded가 없는가.
- tracked= 값이 대부분 0이 아닌가.
- metrics.txt가 생성되는가.

7.4 Rerun 기록 생성

Rerun 빌드를 사용하면 실행 결과를 .rrd로 저장할 수 있다.

```
cd /mnt/d/02_research/04_cpp_LC-EKF_VIO
./build_rerun/run_vio \
  config/kitti_raw_2011_09_26_drive_0117.yaml \
  --rerun-save results/kitti_raw_2011_09_26_drive_0117/vio_rerun.rrd \
  --rerun-image-every 10
```

Listing 7.4: Rerun 파일 저장 실행

--rerun-image-every 0은 이미지 로깅을 끈다. 궤적과 scalar metric만 보고 싶으면 파일 크기를 크게 줄일 수 있다.

제 8 장

결과 검증과 시각화

8.1 출력 파일

파일	의미
trajectory_tum.txt	EKF 추정 위치. TUM 형식
gt_tum.txt	ground truth 위치. GT가 있을 때만 생성
trajectory_aligned_tum.txt	GT에 SE(3) 정렬한 추정 궤적
state_log.txt	position, velocity, Euler, quaternion, feature count
imu_log.txt	사용한 gyro/accel 로그
metrics.txt	frames, ATE RMSE
vio_rerun.rrd	Rerun 선택 실행 시 생성되는 기록 파일

8.2 plot 생성

Python 시각화 도구는 결과 디렉터리를 인자로 받는다.

```
cd /mnt/d/02_research/04_cpp_LC-EKF_VIO
python3 tools/plot_trajectory.py results/kitti_raw_2011_09_26_drive_0117
```

Listing 8.1: trajectory/state plot 생성

생성물:

```
results/kitti_raw_2011_09_26_drive_0117/trajectory_plot.png
results/kitti_raw_2011_09_26_drive_0117/state_plot.png
```

8.3 정상/비정상 판정

관찰값	판단
ATE 1–3 m, trajectory 모 양이 GT와 유사	KITTI 짧은 시퀀스 기준 정상 범위

feature count가 초반부터 0 근처	이미지 경로, rectification, stereo baseline, 조명/텍스처 문제 확인
ATE 10 m 이상 또는 한 방향으로 급발산	T_cam_imu 행렬 순서, baseline 부호, dataset/output 경로 혼동 확인
속도가 비현실적으로 커짐	IMU timestamp 순서, gravity 방향, 초기 정지구간 확인
GT unavailable	EuRoC ground truth 또는 KITTI OXTS 파일 누락 가능

제 9 장

운영 절차 체크리스트

9.1 처음 구축할 때

1. WSL Ubuntu에서 g++, cmake, OpenCV4, Eigen3, yaml-cpp 설치.
2. 저장소 루트가 /mnt/d/02_research/04_cpp_LC-EKF_VIO인지 확인.
3. KITTI 또는 EuRoC 데이터를 reader 구조에 맞게 배치.
4. YAML의 dataset과 output을 현재 경로로 수정.
5. KITTI라면 calib 3종에서 T_cam_imu와 intrinsics 확인.
6. cmake -S . -B build -DCMAKE_BUILD_TYPE=Release 실행.
7. cmake --build build -j"\$(nproc)" 실행.
8. ./build/run_vio config/<run>.yaml 실행.
9. python3 tools/plot_trajectory.py <result_dir> 실행.
10. metrics.txt, trajectory_plot.png, state_plot.png 확인.

9.2 새 KITTI drive를 추가할 때

1. 같은 날짜의 calib 파일이 이미 있으면 재사용한다.
2. 새 *_sync drive 디렉터리만 날짜 폴더 아래에 추가한다.
3. 기존 YAML을 복사해 dataset, output, 필요 시 max_frames만 바꾼다.
4. 날짜가 바뀌면 calib 추출을 다시 수행한다.
5. 짧은 max_frames: 100으로 먼저 실행한 뒤 전체 프레임으로 돌린다.

9.3 새 개발자가 가장 먼저 볼 파일

파일	이유
CMakeLists.txt	의존성, 라이브러리, 실행 파일 타겟 정의
apps/run_vio.cpp	CLI, YAML 로딩, reader 선택, 메인 루프, 로그 저장
include/core/types.hpp	데이터 구조와 camera/IMU 파라미터 정의
include/io/*.hpp	dataset reader가 기대하는 디렉터리 구조
include/ frontend/ stereo_tracker.hpp	stereo VO의 외부 인터페이스
include/ekf/*.hpp	IMU propagation과 LC-EKF update 인터페이스
tools/plot_trajectory.py	결과 파일 형식과 시각화 기준

제 10 장

트러블슈팅

10.1 실행 전 오류

오류 메시지 또는 증상	해결 방향
Fatal: bad file 또는 YAML 로딩 실패	config 경로가 틀렸거나 YAML 문법 오류. 들여쓰기와 콜론 확인
Cannot find KITTI stereo image folders	dataset이 drive root가 아니거나 image_00/data 누락
No KITTI OXTS packets found	oxts/data/*.txt 누락 또는 dataset 경로가 한 단계 어긋남
No camera data loaded	EuRoC/KITTI root 구조가 reader 기대 구조와 다름
Camera config is missing a 4x4 T_cam_imu	cam0 또는 cam1 YAML 행렬 누락/행 개수 오류

10.2 실행 후 성능 문제

증상	확인할 것
프레임은 처리되지만 ATE가 매우 큼	T_cam_imu가 T_{CI} 인지, 역행렬 T_{IC} 를 넣지 않았는지 확인
좌우 feature가 거의 잡히지 않음	cam0/cam1 경로가 바뀌었거나 baseline 부호가 반대일 수 있음
초기부터 자세가 이상함	init.static_window_sec 동안 실제로 정지에 가까운지 확인
plot 스크립트가 import 오류 결과 디렉터리가 엉뚱한 곳에 생김	python3-numpy, python3-matplotlib 설치 확인 YAML 상대 경로가 현재 작업 디렉터리 기준으로 해석됨

10.3 경로 문제를 빠르게 찾는 명령

```

cd /mnt/d/02_research/04_cpp_LC-EKF_VIO
python3 - <<'PY'
from pathlib import Path
import yaml

cfg_path = Path("config/kitti_raw_2011_09_26_drive_0117.yaml")
cfg = yaml.safe_load(cfg_path.read_text())
root = Path(cfg["dataset"])
print("dataset:", root, "exists=", root.exists())
for p in ["image_00/data", "image_01/data", "oxts/data"]:
    q = root / p
    print(p, q.exists(), len(list(q.glob("*"))) if q.exists() else 0)
print("output:", Path(cfg["output"]))
PY

```

Listing 10.1: YAML 경로 대상 존재 여부 확인

위 진단 명령은 PyYAML이 설치되어 있을 때만 동작한다. 설치되어 있지 않다면 `sudo apt install python3-yaml`을 사용하거나 YAML 파일을 직접 열어 경로를 확인한다.

제 11 장

부록: 한 번에 따라 하는 명령

11.1 EuRoC V2_02 빠른 실행

PowerShell에서 WSL 명령을 한 줄로 실행하려면 다음 형태를 사용한다. 저장소 루트는 /mnt/d/02_research/04_cpp_LC-EKF_VIO이다.

```
# Normal build
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && cmake --build build -j$(nproc)'

# Run EuRoC V2_02_medium
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && ./build/run_vio config/euroc_v202.yaml'
```

Listing 11.1: 일반 빌드와 EuRoC V2_02 실행

현재 확인된 euroc_v202.yaml 실행 기준 결과는 전체 2348 프레임 처리, ATE RMSE 약 58.98 cm, 출력 폴더 results/euroc_v202이다.

11.2 EuRoC V2_02 IMU-only 실행

IMU-only 설정은 기존 KITTI 방식과 동일하게 ekf.sigma_vo를 매우 크게 두어 VO 위치 보정 영향을 사실상 끄는 방식이다. 설정 파일은 config/euroc_v202_imuonly.yaml이고, 결과는 results/euroc_v202_imuonly에 저장된다.

```
# Run IMU-only
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && ./build/run_vio config/euroc_v202_imuonly.yaml'

# Check metrics
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && cat results/euroc_v202_imuonly/metrics.txt'

# Compare normal VIO vs IMU-only
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && cat results/euroc_v202/metrics.txt && cat results/euroc_v202_imuonly/metrics.txt'
```

Listing 11.2: IMU-only 실행과 결과 확인

cat은 Linux/WSL에서 파일 내용을 터미널에 출력하는 명령이다. PowerShell의 Get-Content와 같은 용도로 사용한다.

11.3 EuRoC V2_02 Rerun 기록

Rerun을 사용할 때 기존 build_rerun 캐시가 예전 저장소 경로를 묻고 있으면 새 빌드 폴더를 쓰는 편이 안전하다. 아래 예시는 build_rerun_v202를 사용한다.

```
# Rerun build
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && cmake -S . -B build_rerun_v202 -
DCMAKE_BUILD_TYPE=Release -DENABLE_RERUN=ON && cmake --build build_rerun_v202 -j$(nproc)'
```

```
# Record normal VIO
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && ./build_rerun_v202/run_vio config/
euroc_v202.yaml --rerun-save results/euroc_v202/euroc_v202_unified_time.rrd --rerun-image-every
10'
```

```
# Open from PowerShell
rerun D:\02_research\04_cpp_LC-EKF_VIO\results\euroc_v202\euroc_v202_unified_time.rrd
```

Listing 11.3: Rerun 빌드와 일반 VIO 기록

```
# Record IMU-only
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && ./build_rerun_v202/run_vio config/
euroc_v202_imuonly.yaml --rerun-save results/euroc_v202_imuonly/euroc_v202_imuonly.rrd --rerun-
image-every 10'
```

```
# Open from PowerShell
rerun D:\02_research\04_cpp_LC-EKF_VIO\results\euroc_v202_imuonly\euroc_v202_imuonly.rrd
```

Listing 11.4: IMU-only Rerun 기록

11.4 Rerun viewer 팁

항목	확인할 것
timeline	time timeline 사용. 의미는 timestamp - first_camera_timestamp
이미지 엔티티	/camera/left_rectified
feature 엔티티	/camera/left_rectified/tracked_features
궤적 엔티티	/ world/ trajectory_est_aligned, / world/ trajectory_gt
blank view	Reset view, Fit, Focus selected를 사용해 view camera를 재설정

```
python -m pip uninstall -y rerun
python -m pip install --force-reinstall rerun-sdk==0.30.2
rerun --version
```

11.5 KITTI 기본 실행

```
# 1. WSL Ubuntu shell
cd /mnt/d/02_research/04_cpp_LC-EKF_VIO

# 2. Dependencies, first time only
sudo apt update
sudo apt install -y build-essential cmake git pkg-config unzip \
  libeigen3-dev libopencv-dev libyaml-cpp-dev \
  python3 python3-numpy python3-matplotlib

# 3. Edit config before running:
#   dataset: "data/kitti_raw/2011_09_26/2011_09_26_drive_0117_sync"
#   output:  "results/kitti_raw_2011_09_26_drive_0117"

# 4. Build
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j"${nproc}"

# 5. Run
./build/run_vio config/kitti_raw_2011_09_26_drive_0117.yaml

# 6. Plot
python3 tools/plot_trajectory.py results/kitti_raw_2011_09_26_drive_0117
```

Listing 11.6: 처음부터 실행까지

11.6 Rerun 기록 포함 실행

```
cd /mnt/d/02_research/04_cpp_LC-EKF_VIO
cmake -S . -B build_rerun -DCMAKE_BUILD_TYPE=Release -DENABLE_RERUN=ON
cmake --build build_rerun -j"${nproc}"

./build_rerun/run_vio \
  config/kitti_raw_2011_09_26_drive_0117.yaml \
  --rerun-save results/kitti_raw_2011_09_26_drive_0117/vio_rerun.rrd \
  --rerun-image-every 10
```

Listing 11.7: Rerun 선택 실행

11.7 최종 확인

구축 완료 후 다음 네 파일을 확인한다.

```
ls -lh results/kitti_raw_2011_09_26_drive_0117/metrics.txt
ls -lh results/kitti_raw_2011_09_26_drive_0117/trajectory_tum.txt
```

```
ls -lh results/kitti_raw_2011_09_26_drive_0117/trajectory_plot.png
ls -lh results/kitti_raw_2011_09_26_drive_0117/state_plot.png
```

이 매뉴얼은 구축과 재현을 위한 절차서다. 수식 유도과 코드 내부 동작을 더 깊게 보려면 README.md, docs/KITTI_FULL_PIPELINE_GUIDE.md, docs/latex/guide.pdf를 함께 참고한다.